

NBA Shot Make Probability Prediction

Project Writeup

Fadi Qarmash

1. Project Goal

The goal of this project was to predict the probability that an NBA shot would be made. The training data contained 425,719 shot attempts and included information about shot location, the shot clock, defenders, player movement and the type of shot. The required output was a probability between 0 and 1 for each shot in the testing file.

My approach was to clean the tracking data, create usable numerical features, reduce the number of features, compare several classification models, and then use the selected model to create the final make_prob values.

2. Data Preparation

I loaded the compressed training file directly with Polars and then converted it to pandas for the rest of the work. One column, closestdefapproach, contained several defender-distance values inside one field. I split it into four separate numerical features: def_appr_1, def_appr_0_75, def_appr_0_5 and def_appr_0_25.

Missing categorical values were filled with "Unknown" and missing numerical values were filled with the median. The distcont columns were converted to numeric values, booleans were converted to 0/1 values, and the remaining categorical variables such as month, gamestate and shottype were one-hot encoded.

I removed high-cardinality ID fields such as shot_id, shooter_id, team IDs and defender IDs from the model input. This reduced the chance of the models memorising IDs instead of learning more general shot patterns.

3. Feature Selection

To reduce the feature set, I trained a Random Forest on a 50,000-row sample and ranked features by its feature importance. I kept the top 20 features for model training. The strongest ranked features were mostly related to shot position, timing and defensive pressure.

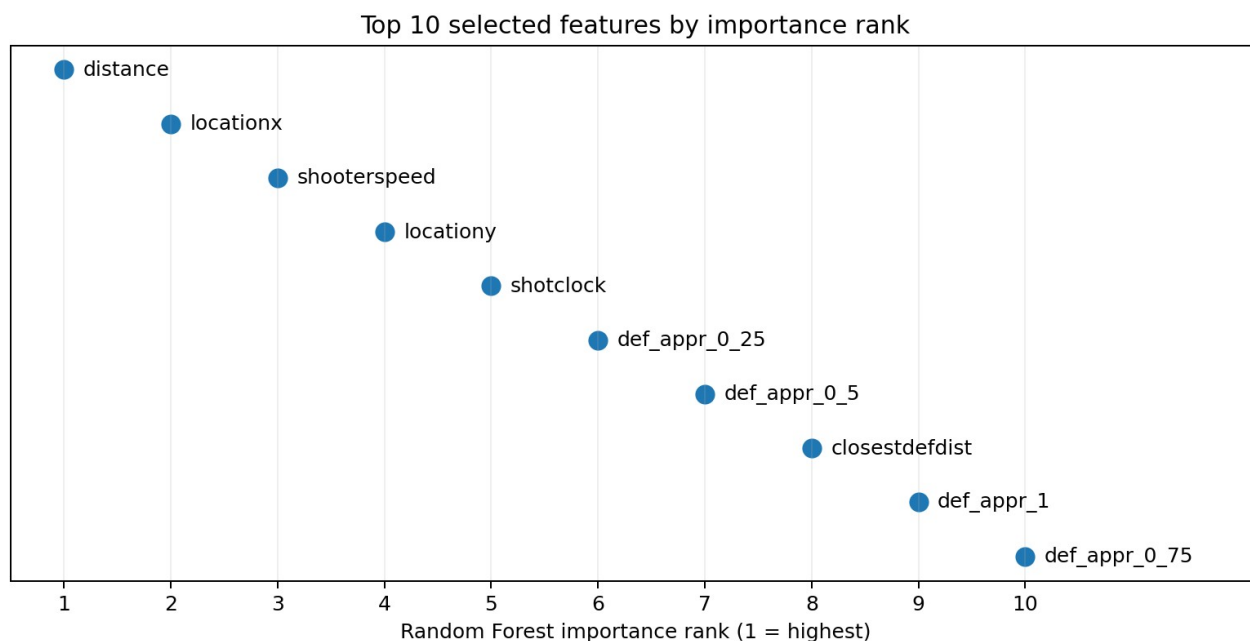


Figure 1. Top ten features in the Random Forest importance ranking. The chart shows rank only, because the notebook stored the ordering but did not print the exact importance values.

Feature group	Examples	What it represents
Shot location	distance, locationx, locationy	Where the shot was taken from on the court.
Timing and movement	shotclock, shooterspeed, dribblesbefore	How much time was available and how the shooter was moving before the shot.
Defensive pressure	closestdefdist, def_appr_*, distcont1, distcont2, num_contesters	How close defenders were and how the defence approached the shooter.
Shot / game context	shottype_jumper, month indicators	Basic context around the shot.

4. Model Development and Iteration

In the first version, I trained Logistic Regression, Random Forest and XGBoost. The full training setup was too heavy for the available memory and runtime, so I reduced the tree settings and trained on a 100,000-row subset. XGBoost was used for the first submission because it gave a reasonable balance of results at that stage.

For the second iteration, I increased the training subset to 250,000 rows and added Extra Trees and HistGradientBoosting. This gave five models to compare using the same validation set and selected features.

First iteration: 100,000 training rows, three models, and XGBoost used for the first submission.

Second iteration: 250,000 training rows, five models after adding Extra Trees and HistGradientBoosting, with Logistic Regression selected for the updated submission.

5. Validation Results

I compared the five models using Accuracy, Precision, Recall and F1-score. HistGradientBoosting had the highest Accuracy and Precision, while Logistic Regression had the highest Recall and F1-score. Because my notebook used F1-score as the final selection rule, Logistic Regression was selected for the updated submission.

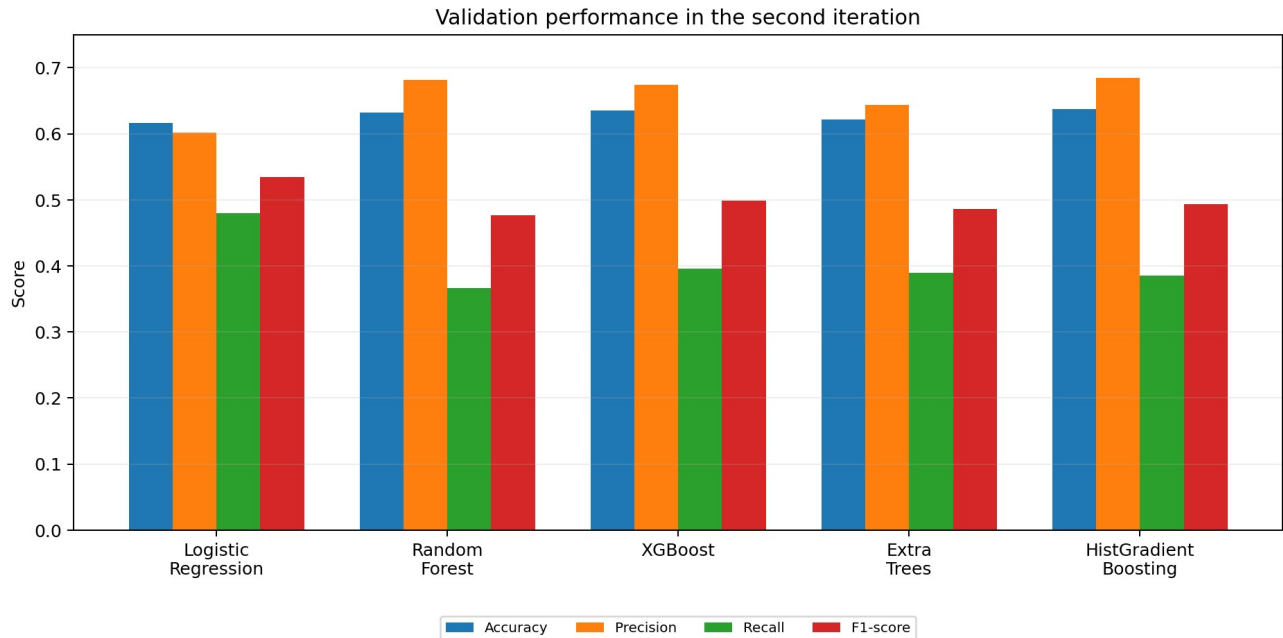


Figure 2. Validation results for the five models in the second iteration.

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.616	0.601	0.480	0.534
Random Forest	0.632	0.682	0.367	0.477
XGBoost	0.635	0.674	0.396	0.499
Extra Trees	0.622	0.644	0.390	0.486
HistGradientBoosting	0.637	0.684	0.385	0.493

6. Final Model and Output

The final notebook selected Logistic Regression because it had the highest validation F1-score, about 0.534. The model used the 20 selected features after scaling with StandardScaler. For the testing data, I repeated the same cleaning and encoding steps, matched the test columns to the training columns, applied the fitted scaler and generated make probabilities with predict_proba().

The final submission file contained two columns: shot_id and make_prob. For example, the first few predicted probabilities in the notebook were 0.3468, 0.6014, 0.5870, 0.5635 and 0.3657 for shot IDs 1, 5, 6, 8 and 10 respectively.

7. Limitations and What I Would Improve

The biggest practical issue was memory usage. Parsing the defender-approach column and training several models on more than 400,000 rows was expensive, so some model training had to use subsets. With more computing resources, I would train and tune the models on a larger part of the training set and use cross-validation instead of relying on one split.

A more important modelling issue is the evaluation metric. The project brief says the submitted probabilities are evaluated using log-loss, but my notebook selected the final model using F1-score, which is based on class predictions rather than probability quality. A better next version would compare models using validation log-loss and tune the model directly for probability performance. I would also check probability calibration, since well-calibrated probabilities matter when log-loss is the scoring metric.

I would also test more features created only from the provided dataset. Examples are interactions between distance and shot type, distance and defender pressure, or shot-clock ranges. Another option would be carefully built historical shooter features from the training data, as long as they are created without leaking information from the validation or test rows. This would follow the project rule that no external data can be used.

Finally, I would compare the simple Logistic Regression result with non-linear models after proper hyperparameter tuning. The second iteration showed that the more complex models did not automatically give the best F1-score, so model complexity by itself is not enough; the evaluation metric and the quality of the features are more important.

8. Final Takeaway

The project went through two clear versions. The first version produced a working XGBoost submission after reducing the model size to fit the available runtime. The second version used more training data, compared two additional models and selected Logistic Regression based on F1-score. The main lesson from the second version was that a simpler model can still compete with tree-based methods when the input features contain useful information. The main improvement I would make next is to select and tune models using log-loss, because that is the metric used to judge the submitted probabilities.